Full Name:	Aishwarya S
Email:	aishwaryasriramu@gmail.com
Test Name:	**Mock Test**
Taken On:	26 Aug 2025 18:48:08 IST
Time Taken:	41 min 4 sec/ 105 min
Invited by:	Ankush
Invited on:	26 Aug 2025 18:47:56 IST
Skills Score:	
Tags Score:	Algorithms 255/255 Core CS 255/255 Data Structures 60/60 Disjoint Set 60/60 Graph Theory 100/100 Medium 195/195 Search 95/95 problem-solving 195/195

100%

255/255

scored in **Mock Test** in 41 min 4 sec on 26 Aug 2025 18:48:08 IST

Recruiter/Team Comments:

No Comments.

	Question Description	Time Taken	Score	Status
Q1	Breadth First Search: Shortest Reach > **Coding**	1 min 26 sec	100/ 100	✓
Q2	Components in a graph > **Coding**	40 sec	60/ 60	✓
Q3	Cut the Tree > **Coding**	41 sec	95/ 95	✓

QUESTION 1

Correct Answer

Score 100

Breadth First Search: Shortest Reach > Coding

Graph Theory | Algorithms | Medium | problem-solving | Core CS

QUESTION DESCRIPTION

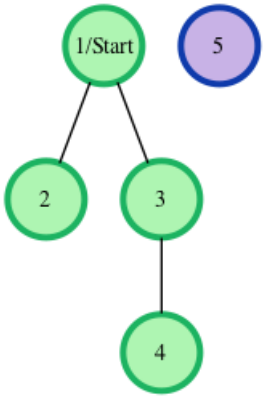
Consider an undirected graph where each edge weighs 6 units. Each of the nodes is labeled consecutively from 1 to n.

You will be given a number of queries. For each query, you will be given a list of edges describing an undirected graph. After you create a representation of the graph, you must determine and report the shortest distance to each of the other nodes from a given starting position using the *breadth-first search*

algorithm (BFS). Return an array of distances from the start node in node number order. If a node is unreachable, return -1 for that node.

Example

The following graph is based on the listed inputs:



$n = 5$ // number of nodes
$m = 3$ // number of edges
$edges = [1, 2], [1, 3], [3, 4]$
$s = 1$ // starting node

All distances are from the start node 1. Outputs are calculated for distances to nodes 2 through 5: $[6, 6, 12, -1]$. Each edge is 6 units, and the unreachable node 5 has the required return distance of -1.

Function Description

Complete the *bfs* function in the editor below. If a node is unreachable, its distance is -1.

bfs has the following parameter(s):

- *int n*: the number of nodes
- *int m*: the number of edges
- *int edges[m][2]*: start and end nodes for edges
- *int s*: the node to start traversals from

Returns

int[n-1]: the distances to nodes in increasing node number order, not including the start node (-1 if a node is not reachable)

Input Format

The first line contains an integer q, the number of queries. Each of the following q sets of lines has the following format:

- The first line contains two space-separated integers n and m, the number of nodes and edges in the graph.
- Each line i of the m subsequent lines contains two space-separated integers, u and v, that describe an edge between nodes u and v.
- The last line contains a single integer, s, the node number to start from.

Constraints

- $1 \leq q \leq 10$
- $2 \leq n \leq 1000$
- $1 \leq m \leq \frac{n \cdot (n-1)}{2}$
- $1 \leq u, v, s \leq n$

Sample Input

```
2
4 2
1 2
1 3
1
3 1
```

```
2 3
2
```

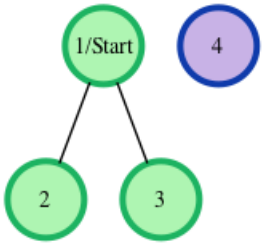
Sample Output

```
6 6 -1
-1 6
```

Explanation

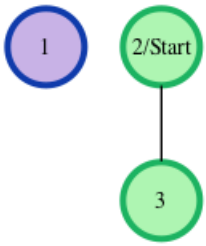
We perform the following two queries:

1. The given graph can be represented as:



where our *start* node, s, is node 1. The shortest distances from s to the other nodes are one edge to node 2, one edge to node 3, and an infinite distance to node 4 (which it is not connected to). We then return an array of distances from node 1 to nodes 2, 3, and 4 (respectively): $[6, 6, -1]$.

2. The given graph can be represented as:



where our *start* node, s, is node 2. There is only one edge here, so node 1 is unreachable from node 2 and node 3 has one edge connecting it to node 2. We then return an array of distances from node 2 to nodes 1, and 3 (respectively): $[-1, 6]$.

Note: Recall that the actual length of each edge is 6, and we return -1 as the distance to any node that is unreachable from s.

CANDIDATE ANSWER

Language used: **C**

```
1
2  /*
3   * Complete the 'bfs' function below.
4   *
5   * The function is expected to return an INTEGER_ARRAY.
6   * The function accepts following parameters:
7   *  1. INTEGER n
8   *  2. INTEGER m
9   *  3. 2D_INTEGER_ARRAY edges
10  *  4. INTEGER s
11  */
12
13 /*
14  * To return the integer array from the function, you should:
15  *     - Store the size of the array to be returned in the result_count
16 variable
17  *     - Allocate the array statically or dynamically
```

```
 *
 * For example,
 * int* return_integer_array_using_static_allocation(int* result_count) {
 *     *result_count = 5;
 *
 *     static int a[5] = {1, 2, 3, 4, 5};
 *
 *     return a;
 * }
 *
 * int* return_integer_array_using_dynamic_allocation(int* result_count) {
 *     *result_count = 5;
 *
 *     int *a = malloc(5 * sizeof(int));
 *
 *     for (int i = 0; i < 5; i++) {
 *         *(a + i) = i + 1;
 *     }
 *
 *     return a;
 * }
 *
 */

#define EDGE_WEIGHT 6

typedef struct Node{
    int vertex;
    struct Node* next;
}Node;

Node* adj[1001];
int visited[1001];
int dist[1001];

int queue[1001];
int front, rear;

void enqueue(int x){
    queue[rear++] = x;
}

int dequeue(){
    return queue[front++];
}

int isEmpty(){
    return front == rear;
}

void addEdge(int u, int v){
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = v;
    newNode->next = adj[u];
    adj[u] = newNode;

    newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = u;
    newNode->next = adj[v];
    adj[v] = newNode;
}
```

```
int* bfs(int n, int m, int edges_rows, int edges_columns, int** edges, int s,
int* result_count) {

for(int i=1;i<=n;i++){
    adj[i] = NULL;
}

for(int i=0;i<edges_rows;i++){
    int u = edges[i][0];
    int v = edges[i][1];
    addEdge(u,v);
}

for(int i=1;i<=n;i++){
    visited[i] = 0;
    dist[i] = -1;
}

front = rear= 0;

visited[s] = 1;
dist[s] = 0;
enqueue(s);

while(!isEmpty()){
    int u = dequeue();
    Node* temp = adj[u];
    while(temp != NULL){
        int v = temp->vertex;
        if(!visited[v]){
            visited[v] = 1;
            dist[v] = dist[u] + EDGE_WEIGHT;
            enqueue(v);
        }
        temp = temp->next;
    }
}
*result_count = n-1;
int* result = (int*)malloc((*result_count) * sizeof(int));
int idx = 0;
for(int i=1;i<=n;i++){
    if(i==s) continue;
    result[idx++] = dist[i];
}
return result;
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Easy	Sample case	Success	0	0.0076 sec	7 KB
Testcase 2	Medium	Hidden case	Success	5	0.0085 sec	7.38 KB
Testcase 3	Medium	Hidden case	Success	5	0.0251 sec	14.1 KB
Testcase 4	Hard	Hidden case	Success	15	0.0085 sec	7.38 KB
Testcase 5	Hard	Hidden case	Success	15	0.0095 sec	7.63 KB
Testcase 6	Hard	Hidden case	Success	30	0.1194 sec	46.4 KB
Testcase 7	Hard	Hidden case	Success	30	0.0133 sec	8.63 KB
Testcase 8	Easy	Sample case	Success	0	0.0083 sec	7.25 KB

No Comments

QUESTION 2

Correct Answer

Score 60

Components in a graph › Coding

Algorithms | Data Structures | Disjoint Set | Core CS

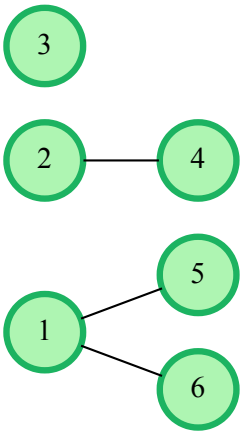
QUESTION DESCRIPTION

There are $2 \times N$ nodes in an undirected graph, and a number of edges connecting some nodes. In each edge, the first value will be between 1 and N, inclusive. The second node will be between $N+1$ and $2 \times N$, inclusive. Given a list of edges, determine the size of the smallest and largest connected components that have 2 or more nodes. A node can have any number of connections. The highest node value will always be connected to at least 1 other node.

Note Single nodes should not be considered in the answer.

Example

$bg = [[1, 5], [1, 6], [2, 4]]$



The smaller component contains 2 nodes and the larger contains 3. Return the array $[2, 3]$.

Function Description

Complete the *connectedComponents* function in the editor below.

connectedComponents has the following parameter(s):
- *int bg[n][2]:* a 2-d array of integers that represent node ends of graph edges

Returns

- *int[2]:* an array with 2 integers, the smallest and largest component sizes

Input Format

The first line contains an integer n, the size of bg.
Each of the next n lines contain two space-separated integers, $bg[i][0]$ and $bg[i][1]$.

Constraints

- $1 \leq numberofnodesN \leq 15000$
- $1 \leq bg[i][0] \leq N$
- $N+1 \leq bg[i][1] \leq 2N$

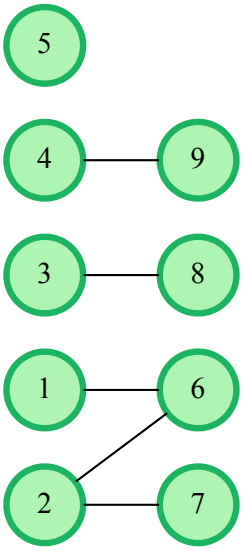
Sample Input

```
STDIN   Function
-----   --------
5       bg[] size n = 5
1 6     bg = [[1, 6],[2, 7], [3, 8], [4,9], [2, 6]]
2 7
3 8
4 9
2 6
```

Sample Output

```
2 4
```

Explanation



Since the component with node 5 contains only one node, it is not considered.

The number of vertices in the smallest connected component in the graph is 2 based on either $(3, 8)$ or $(4, 9)$.

The number of vertices in the largest connected component in the graph is 4 i.e. $1 - 2 - 6 - 7$.

CANDIDATE ANSWER

Language used: **C**

```c

/*
 * Complete the 'componentsInGraph' function below.
 *
 * The function is expected to return an INTEGER_ARRAY.
 * The function accepts 2D_INTEGER_ARRAY gb as parameter.
 */

/*
 * To return the integer array from the function, you should:
 *     - Store the size of the array to be returned in the result_count
variable
 *     - Allocate the array statically or dynamically
 *
 * For example,
 * int* return_integer_array_using_static_allocation(int* result_count) {
 *     *result_count = 5;
 *
 *     static int a[5] = {1, 2, 3, 4, 5};
 *
 *     return a;
 * }
 *
 * int* return_integer_array_using_dynamic_allocation(int* result_count) {
 *     *result_count = 5;
 *
 *     int *a = malloc(5 * sizeof(int));
```

```
 *
 *     for (int i = 0; i < 5; i++) {
 *         *(a + i) = i + 1;
 *     }
 *
 *     return a;
 * }
 *
 */

#define MAXN 30010

int parent[MAXN];
int size[MAXN];

int find(int x){
    if(parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void unite(int a, int b){
    int pa = find(a);
    int pb = find(b);
    if(pa == pb) return;

    if(size[pa] < size[pb]){
        parent[pa] = pb;
        size[pb] += size[pa];
    }else{
        parent[pb] = pa;
        size[pa] += size[pb];
    }
}

int* componentsInGraph(int gb_rows, int gb_columns, int** gb, int*
result_count) {
static int result[2];
int maxNode = 0;

for(int i=0; i<MAXN; i++){
    parent[i] = i;
    size[i] =1;
}

for(int i=0;i<gb_rows; i++){
    int u = gb[i][0];
    int v = gb[i][1];
    unite(u, v);
    if(u>maxNode) maxNode = u;
    if(v>maxNode) maxNode = v;
}
int minComp = MAXN;
int maxComp = 0;

for(int i=1; i<=maxNode; i++){
    if(parent[i] == i && size[i] > 1){
        if(size[i] < minComp) minComp = size[i];
        if(size[i] > maxComp) maxComp = size[i];
    }
}

result[0] = minComp;
```

```
result[1] = maxComp;
*result_count = 2;
return result;
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Medium	Hidden case	Success	0	0.0075 sec	7.38 KB
Testcase 2	Medium	Hidden case	Success	0	0.0089 sec	7.5 KB
Testcase 3	Medium	Hidden case	Success	0	0.008 sec	7.25 KB
Testcase 4	Medium	Hidden case	Success	0	0.0088 sec	7.25 KB
Testcase 5	Medium	Hidden case	Success	0	0.0115 sec	7.63 KB
Testcase 6	Medium	Hidden case	Success	0	0.0108 sec	7.38 KB
Testcase 7	Medium	Hidden case	Success	0	0.0118 sec	7.75 KB
Testcase 8	Medium	Hidden case	Success	0	0.0092 sec	7.88 KB
Testcase 9	Medium	Hidden case	Success	0	0.0129 sec	7.88 KB
Testcase 10	Medium	Hidden case	Success	0	0.0092 sec	7.75 KB
Testcase 11	Medium	Hidden case	Success	0	0.0145 sec	7.63 KB
Testcase 12	Medium	Hidden case	Success	0	0.0222 sec	7.88 KB
Testcase 13	Medium	Hidden case	Success	0	0.0087 sec	8 KB
Testcase 14	Medium	Hidden case	Success	0	0.0088 sec	8 KB
Testcase 15	Medium	Hidden case	Success	0	0.0137 sec	8 KB
Testcase 16	Medium	Hidden case	Success	0	0.0099 sec	8.38 KB
Testcase 17	Medium	Hidden case	Success	0	0.0095 sec	7.5 KB
Testcase 18	Medium	Hidden case	Success	0	0.011 sec	8.25 KB
Testcase 19	Easy	Sample case	Success	0	0.0073 sec	7.38 KB
Testcase 20	Medium	Hidden case	Success	0	0.0135 sec	8.75 KB
Testcase 21	Medium	Hidden case	Success	0	0.0169 sec	8.88 KB
Testcase 22	Medium	Hidden case	Success	0	0.0139 sec	9 KB
Testcase 23	Medium	Hidden case	Success	0	0.0197 sec	8.75 KB
Testcase 24	Medium	Hidden case	Success	0	0.0133 sec	8.88 KB
Testcase 25	Medium	Hidden case	Success	0	0.0123 sec	9 KB
Testcase 26	Medium	Hidden case	Success	0	0.0165 sec	8.63 KB
Testcase 27	Medium	Hidden case	Success	0	0.0125 sec	8.63 KB
Testcase 28	Medium	Hidden case	Success	0	0.0142 sec	8.63 KB
Testcase 29	Medium	Hidden case	Success	0	0.0118 sec	8.75 KB
Testcase 30	Medium	Hidden case	Success	0	0.0117 sec	8.88 KB
Testcase 31	Medium	Hidden case	Success	0	0.013 sec	8.63 KB
Testcase 32	Medium	Hidden case	Success	0	0.024 sec	8.75 KB
Testcase 33	Medium	Hidden case	Success	0	0.012 sec	8.63 KB
Testcase 34	Hard	Hidden case	Success	10	0.0121 sec	8.88 KB
Testcase 35	Hard	Hidden case	Success	10	0.0168 sec	8.5 KB
Testcase 36	Hard	Hidden case	Success	10	0.0116 sec	8.75 KB

Testcase 37	Hard	Hidden case	Success	10	0.0132 sec	9 KB
Testcase 38	Hard	Hidden case	Success	10	0.0111 sec	8.75 KB
Testcase 39	Hard	Hidden case	Success	10	0.0126 sec	8.75 KB

No Comments

QUESTION 3

Correct Answer

Score 95

Cut the Tree › Coding

Search | Algorithms | Medium | problem-solving | Core CS

QUESTION DESCRIPTION

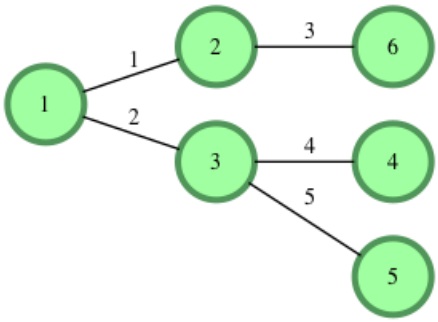
There is an undirected tree where each vertex is numbered from 1 to n, and each contains a data value. The *sum* of a tree is the sum of all its nodes' data values. If an edge is cut, two smaller trees are formed. The *difference* between two trees is the absolute value of the difference in their sums.

Given a tree, determine which edge to cut so that the resulting trees have a minimal *difference* between them, then return that difference.

Example

$data = [1, 2, 3, 4, 5, 6]$
$edges = [(1, 2), (1, 3), (2, 6), (3, 4), (3, 5)]$

In this case, node numbers match their weights for convenience. The graph is shown below.



The values are calculated as follows:

```
Edge    Tree 1  Tree 2  Absolute
Cut     Sum      Sum     Difference
1        8         13         5
2        9         12         3
3        6         15         9
4        4         17        13
5        5         16        11
```

The minimum absolute difference is 3.

Note: The given tree is *always* rooted at vertex 1.

Function Description

Complete the *cutTheTree* function in the editor below.

cutTheTree has the following parameter(s):

- *int data[n]:* an array of integers that represent node values
- *int edges[n-1][2]:* an 2 dimensional array of integer pairs where each pair represents nodes connected by the edge

Returns

- *int:* the minimum achievable absolute difference of tree sums

Input Format

The first line contains an integer n, the number of vertices in the tree.
The second line contains n space-separated integers, where each integer u denotes the $node[u]$ data value, $data[u]$.
Each of the $n-1$ subsequent lines contains two space-separated integers u and v that describe edge $u \leftrightarrow v$ in tree t.

Constraints

- $3 \leq n \leq 10^5$
- $1 \leq data[u] \leq 1001$, where $1 \leq u \leq n$.

Sample Input

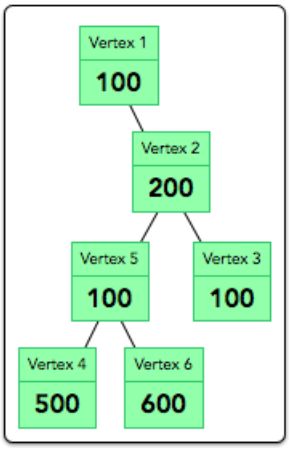
```
STDIN                       Function
-----                       --------
6                           data[] size n = 6
100 200 100 500 100 600     data = [100, 200, 100, 500, 100, 600]
1 2                         edges = [[1, 2], [2, 3], [2, 5], [4, 5], [5,
6]]
2 3
2 5
4 5
5 6
```

Sample Output

```
400
```

Explanation

We can visualize the initial, uncut tree as:



There are $n-1=5$ edges we can cut:

1. Edge $1 \leftrightarrow 2$ results in $d_{1 \leftrightarrow 2} = 1500 - 100 = 1400$
2. Edge $2 \leftrightarrow 3$ results in $d_{2 \leftrightarrow 3} = 1500 - 100 = 1400$
3. Edge $2 \leftrightarrow 5$ results in $d_{2 \leftrightarrow 5} = 1200 - 400 = 800$
4. Edge $4 \leftrightarrow 5$ results in $d_{4 \leftrightarrow 5} = 1100 - 500 = 600$
5. Edge $5 \leftrightarrow 6$ results in $d_{5 \leftrightarrow 6} = 1000 - 600 = 400$

The minimum *difference* is 400.

CANDIDATE ANSWER

Language used: **C**

```
1  /*
2   * Complete the 'cutTheTree' function below.
3   *
4   * The function is expected to return an INTEGER.
```

```
 * The function accepts following parameters:
 *  1. INTEGER_ARRAY data
 *  2. 2D_INTEGER_ARRAY edges
 */
#define MAXN 100001

int data_ptr[MAXN];
long long totalSum;
long long subtreeSum[MAXN];
int *adj[MAXN];
int adjSize[MAXN];
int adjCap[MAXN];
int visited[MAXN];

void addEdge(int u, int v){
    if(adjSize[u] == adjCap[u]){
        adjCap[u] = adjCap[u] == 0 ? 2: adjCap[u] * 2;
        adj[u] = realloc(adj[u], adjCap[u] * sizeof(int));
    }
    adj[u][adjSize[u]++] = v;
}

long long dfs(int u){
    visited[u] = 1;
    long long sum = data_ptr[u];

    for(int i=0;i<adjSize[u]; i++){
        int v = adj[u][i];
        if(!visited[v]){
            sum+=dfs(v);
        }
    }
    subtreeSum[u] = sum;
    return sum;
}

int cutTheTree(int data_count, int* data, int edges_rows, int edges_columns,
int** edges) {
int n = data_count;
totalSum = 0;

for(int i=1;i<=n; i++){
    data_ptr[i] = data[i-1];
    totalSum+=data_ptr[i];
    adj[i] = NULL;
    adjSize[i] = 0;
    adjCap[i] = 0;
    visited[i] = 0;
}

for(int i=0;i<edges_rows;i++){
    int u = edges[i][0];
    int v = edges[i][1];
    addEdge(u,v);
    addEdge(v,u);
}

dfs(1);

long long minDiff = LLONG_MAX;
for(int i=2;i<=n;i++){
    long long s1 = subtreeSum[i];
    long long s2 = totalSum - s1;
```

```
    long long diff = llabs(s1-s2);
    if(diff<minDiff){
        minDiff = diff;
    }
}
return (int)minDiff;
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Easy	Sample case	Success	0	0.0073 sec	7.25 KB
Testcase 2	Hard	Hidden case	Success	5	0.0091 sec	7.13 KB
Testcase 3	Hard	Hidden case	Success	5	0.0072 sec	7.13 KB
Testcase 4	Hard	Hidden case	Success	5	0.0075 sec	7 KB
Testcase 5	Easy	Sample case	Success	0	0.0098 sec	7.13 KB
Testcase 6	Hard	Hidden case	Success	5	0.0142 sec	9.13 KB
Testcase 7	Hard	Hidden case	Success	10	0.0577 sec	25.4 KB
Testcase 8	Hard	Hidden case	Success	5	0.0577 sec	25.5 KB
Testcase 9	Hard	Hidden case	Success	5	0.0614 sec	25.3 KB
Testcase 10	Hard	Hidden case	Success	5	0.0635 sec	25.8 KB
Testcase 11	Hard	Hidden case	Success	5	0.055 sec	25.6 KB
Testcase 12	Hard	Hidden case	Success	5	0.059 sec	25.3 KB
Testcase 13	Medium	Hidden case	Success	5	0.0665 sec	25 KB
Testcase 14	Medium	Hidden case	Success	5	0.0472 sec	25.3 KB
Testcase 15	Medium	Hidden case	Success	5	0.061 sec	25 KB
Testcase 16	Medium	Hidden case	Success	5	0.0666 sec	25.6 KB
Testcase 17	Medium	Hidden case	Success	5	0.0613 sec	25.5 KB
Testcase 18	Medium	Hidden case	Success	5	0.059 sec	25.8 KB
Testcase 19	Medium	Hidden case	Success	5	0.0639 sec	24.8 KB
Testcase 20	Medium	Hidden case	Success	5	0.0499 sec	25.5 KB

No Comments